UBAS. Unified BIDS Automation at Scale

(with whole-brain modeling demonstration)

Isaac Reyes-Gonzalez[1]

[1]Neuroscience Training Program. University of Wisconsin-Madison

2025-12-10

1 Introduction

UBAS (*Unified BIDS Automation at Scale*, https://github.com/isacdaavid/ubas) is a Python library to manipulate BIDS datasets using modern, scalable and general-purpose declarative interfaces. It allows researchers to concentrate on the neuroscience and the peculiarities of their analyses, while UBAS handles data loading, efficient representation and parallel processing. Here we explain and demonstrate its workings using a functional and diffusion MRI dataset, which we use to run a whole-brain dynamical model of resting-state BOLD activity.

The BIDS standard (*Brain Imaging Data Structure*) is being increasingly adopted to store, share and even preprocess all sorts of neuroimaging data (K. J. Gorgolewski et al. 2016). At its core, it defines conventions to organize neuroimaging datasets in a file system. However, flexible and efficient tools to help researchers implement their in-house studies starting from a BIDS folder are lacking. Software containers like `fmriprep`, `qsiprep`, `xcp-d` and `qsirecon` have been built to cover common preprocessing steps in a BIDS-aware manner (Esteban et al. 2019; Cieslak et al. 2021; Mehta et al. 2024). These tools in turn rely on a plethora of more primitive methods coming from previous neuroimaging analysis packages. In spite of such automatizations, virtually all studies reaching this pre-analysis stage will require further manual interaction with the dataset or its derivatives, in order to complete their specific computational goals and answer their research questions.

Instead of shipping prefabricated analysis pipelines for common use cases (like clinica, Routier et al. 2021), the UBAS philosophy fits well with researchers who want control over their heterogeneous analyses, but do not want to waste time writing error-prone loops to load and manipulate the data from scratch. It also appeals to researchers who would like to further standardize adoption of BIDS, and improve reproducibility while exploiting UBAS scalability capabilities as a computational engine. In fact, UBAS implements *no* neuroimaging analysis

methods at all. Rather, it stands as an intermediary interface between the data and the methods, and aims for compatibility with existing algorithmic arsenals (e.g. via the `nibabel`, `nipype` and `nilearn` aggregator interfaces Brett et al. 2024; K. Gorgolewski et al. 2011).

In other words, UBAS provides *meta-computational* infrastructure tailored around BIDS.

2 Methods

2.1 The library

UBAS provides four main features:

- **BIDS transparency**: UBAS objects follow the same intuitive hierarchy as the BIDS data in the file system: cohorts containing subjects, containing sessions, containing modalities, containing files and metadata, etc.
- **Data parallelism**: computation is accelerated by default using proven primitives from the big data field: collecting, storing, filtering and processing in parallel is as easy as defining how to do it for one element.
- **Memory efficient**: Achieved via lazy evaluation. File contents are only loaded to RAM when needed. An in-memory cache is still provided to speed-up access.
- **Seamless flexibility**: The same features are available across levels of the tree structure, whether cohorts, subjects, sessions, etc.

The library is implemented using a layer of concrete classes built on top of abstract ones. Concrete classes are meant to represent BIDS directories and files. Meanwhile, all concrete classes provided by UBAS derive common functionality from one or both abstract classes:

`Collection` and `Member`. For instance, a `Subject` can be simulataneously a member of a `Cohort` and a collection of `Session`'s. The following interfaces therefore are available for all kinds of objects in UBAS, although here we only demonstrate them for cohorts and subjects for brevity.

UBAS builds on `pybids` (Yarkoni et al. 2019), which provides primitives to traverse and query the BIDS file structure. UBAS then adds the capacity to represent the actual neuroimaging dataset in a dependable and transparent structure. When a `pybids` layout is passed to one of the UBAS object constructors, it finds its immediate children in the BIDS tree and the process is repeated recursively, until all objects are well-instantiated and become available for the construction of the original collection.

```
>>> from bids import BIDSLayout
>>> layout = BIDSLayout('./data/')

>>> import ubas
>>> mycohort = ubas.Cohort('mycohort', layout)

>>> mycohort
    Cohort('mycohort', subjects={'A00023330', ...})
```

Figure 1: Loading the dataset (represented by the `mycohort` variable).

Collections behave like Python sets supercharged with capabilities like indexing by member label in $O(1)$ time. The type of an object and its label are used to define element uniqueness efficiently. Collection subsets can be derived by calling `.filter()` and passing an arbitrary function describing the condition to be met for single members. This is syntactic sugar for `subcohort = ubas.Cohort(filter(condition, cohort))`. Arbitrary member attributes can be easily collected for the whole collection by calling the lazy generator `.collect()`, or overwritten using `.store()`. Both methods support attributes deep-nested within members.

The workhorse of parallel processing is provided by `.compute()`. This is a parallel implementation of the `map` function (also known as `apply`). Similarly to `.filter()`, parallel computation can be achieved by simply passing a user-defined function which operates on single members, together with variable keywords parameters to be passed-through to the function itself. As results are computed by different CPU threads, they may be automatically stored for each member in dictionary called `.quantities` for later retrieval, accessed by the name of the computed function by default.

Although less relevant to end users, similar methods which operate on single objects (as members) are offered for completeness: `collect_self()`, `store_self()` and `compute_self()`. UBAS also uses them internally, to distribute operations from collections to members when it makes sense to.

2.2 Data

We used a publicly available dataset by the Center for Biomedical Research Excellence (COBRE. Aine et al. 2017). The COBRE dataset remains available through the International Neuroimaging Data-sharing Initiative (INDI) via NITRC (https://fcon_1000.projects.nitrc.org/indi/retro/cobre.html). Healthy controls (HC) and patients with schizophrenia spectrum disorder (SSD) were sourced from this dataset. SSD encompasses a range of mental health conditions characterized by delusions, hallucinations, disorganized thinking and abnormal behavior. We considered schizophrenia proper and schizoaffective disorder, which are diagnosed differentially only by heightened concomitant mood disorder symptoms in the case of schizoaffective disorder. Only baseline recordings were included for analysis. Motion-related exclusion was applied using a framewise displacement threshold of 0.5 mm to define outlier time points. Subjects with more than 50% of frames classified as motion outliers were excluded, for a total of 37 patients and

52 controls.

```
>>> patients = mycohort.subjects.filter(patient_list)
>>> controls = mycohort.subjects.filter(control_list)

>>> patients = patients.subjects.filter(low_motion)
>>> controls = controls.subjects.filter(low_motion)

>>> f'{len(patients)} patients and {len(controls)} controls.'
37 patients and 52 controls.
```

Figure 2: Demonstration of the `filter()` functionality to separate patients from controls, and to exclude subjects with more than 50% of high-motion volumes.

2.3 Imaging parameters

COBRE data was originally acquired using a Siemens 3 T TIM Trio MRI scanner using a 12-channel head coil. The resting-state functional MRI (rs-fMRI) sequence consisted of a conventional gradient-echo echoplanar pusle sequence of 150 time points (5 min at a TR of 2 s). The diffusion-weighted MRI sequence (dMRI) sampled 30 different gradient directions at a $b = 800s/mm^2$. The $b = 0$ baseline was measured five times, interspersed between the 30 directional acquisitions. In addition, T1w and T2w scans of head anatomy are included. Sequence parameters of all four image modalities are detailed in Table 1.

Parameter	fMRI	dMRI	T1w	T2w
TR (ms)	2000	9000	2530	15500
TE (ms)	29	84 ms	(multi-echo)	77
TI (ms)			900	
Flip angle	75°		7°	155°
Voxel size (mm)	3.75×3.75×4.55	2x2x2	1x1x1	1.1×1.1×1.5
Slices	32	72	176	120
Slice FOV	64x64	128x128	256x256	192x192

2.4 Preprocessing

rs-fMRI and anatomical preprocessing was performed using fMRIPrep 23.1.4 (Esteban et al. 2019), followed by post-processing using XCP-D (Mehta et al. 2024). dMRI preprocessing was performed using QSIPrep (Cieslak et al. 2021), followed by post-processing using QSIrecon (Cieslak et al. 2021).

In short, fMRIprep performs intensity correction and skull stripping on the T1w images, followed by brain tissue segmentation into cerebrospinal fluid (CSF), white matter (WM) and gray matter (GM) compartments. From these, the cortical surface is reconstructed by FreeSurfer and transformed into the standard surface space *fsaverage* using nonlinear registration, which in turn become part of a grayordinate file (CIFTI) containing 91k samples (approximately 32k cortical vertices per cortical hemisphere, plus subcortical data). For each BOLD functional MRI run, it generated a reference volume and its skull-stripped version. Head-motion parameters were estimated and corrected using transformation matrices and six rotation/translation parameters. Slice-time correction was applied to align the timing of slices within each volume. The BOLD time-series were then resampled to their original native space, accounting for head-motion, resulting in preprocessed BOLD data. The BOLD reference was co-registered to the T1-weighted anatomical reference using boundary-based registration with six degrees of freedom. Confounding time-series were calculated, including framewise displacement, along with global signals from CSF, WM, and whole-brain masks. Finally, the BOLD time-series were resampled into standard space and onto a symmetric template (fsLR) in a CIFTI file with 91k time series. All resamplings were performed in a single interpolation step.

XCP-D performed post-processing on fMRIPrep outputs using the *linc* mode. The pipeline included despiking to remove extreme BOLD values. Nuisance regression was applied using confounds derived from cerebrospinal fluid (CSF), white matter signals, framewise displace-

ment, and six rigid-body motion parameters with their first derivatives. A bandpass filter (0.01–0.1 Hz, 5th order) was applied to retain low-frequency signals. Spatial smoothing was performed with a 3 mm FWHM kernel. Volumes with a framewise displacement exceeding 0.5 mm were censored (scrubbed), and no motion-based filtering (notch or low-pass) was applied. The final output was the censored, cleaned timeseries in CIFTI format. Subsequently, cleaned BOLD data were parcellated using the XCP-D-custom 4S156 parcellation, which combines the Schaefer 2018 cortical atlas (100 regions, Schaefer et al. 2018) with basal ganglia, cerebellar, thalamic and amygdala-hippocampal atlases (Pauli, Nili, and Tyszka 2018; King et al. 2019; Najdenovska et al. 2018).

The QSIprep pipeline applied MP-PCA to denoise diffusion-weighted images. This was followed by distortion correction due to magnetic susceptibility, by eddy currents, and head motion-induced distortions. A single $b = 0$ template per subject was generated. It ensured proper alignment between the diffusion data and an skull-stripped anatomical reference image (T1w). Gradient orientations were reoriented following motion correction. Results are registered to a standard MNI152 space.

QSIrecon performed FOD reconstruction using `singleshell_ss3t_ACT-hsvs`, an `MRItrix3` pipeline. The former employs Anatomically Constrained Tractography (ACT) with the high-order fiber orientation distribution model from the SS3T project and a single-shell 3-tissue (HSVS) algorithm for multi-component fiber response and spherical deconvolution. Anatomical priors from the FreeSurfer outputs were integrated to constrain the tractography. Whole-brain probabilistic tractography was then run to estimate structural connectivity. Finally, streamline connectivity matrices were generated by cross-referencing the tractography results with the same 4S156 parcellation used for functional connectivity. Raw streamline count, SIFT-weighted count and streamline length matrices were produced.

2.5 Whole-brain simulation

Each cortical node in the 4S156 atlas was endowed with a bivariate dynamical system called ALN (adaptive linear-non-linear integrate and fire. Caglar Cakan and Obermayer 2020): a mean-field description of a spiking neural network with both excitatory and inhibitory populations (the AdEx model, more specifically. Brette and Gerstner 2005). The simplification from several single integrate-and-fire neurons to population means is achieved with the Fokker-Planck equation: a partial differential equation to describe the time evolution of the probability distribution of single-neuron firing, subject to local interactions and noise. With the right choice of parameters and under sufficient and balanced excitatory and inhibitory input, the phase space of an ALN neural mass displays bistability around two attractors: one for a down state (low firing) and another for an up state (high firing).

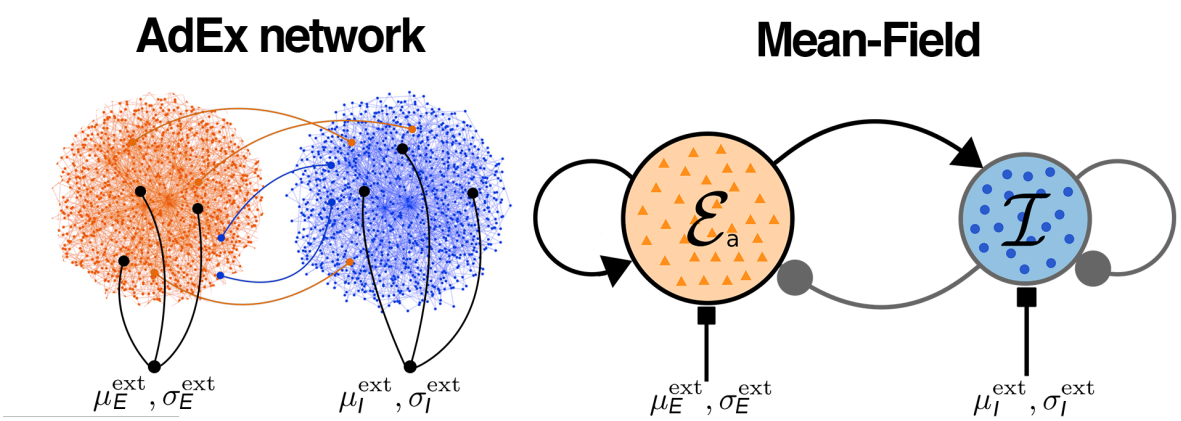

Figure 3: Left: Populations of adaptive exponential integrate-and-fire neurons. Right: Reduction to a mean-field neural mass model (ALN model). Adapted from Caglar Cakan and Obermayer (2020).

For each subject, cortical regions were coupled in a symmetrical fashion, by summing their original structural connectivity matrix C with its transpose C^T and normalizing by dividing all entries by the maximum. Inputs delays are obtained directly from the length matrix D. A very general description of whole-brain models (including global coupling strength constant $\mathbf{g}$ and irrespective of choice of node dynamics $\mathbf{f}$ and type of noise $\boldsymbol{\mu}$) is given by the following

sketch of a system of ordinary differential equations. Each vector $\mathbf{x}_i$ represents a potentially multi-valued state of a node:

$$\frac{d}{dt}\mathbf{x}_i(t) = \underbrace{\mathbf{f}(\mathbf{x}_i(t))}_{local} + \underbrace{\sum_{j=0}^{N} \mathbf{g}\left(C_{ij}, \mathbf{x}_i(t), \mathbf{x}_j\left(t - D_{ij}\right)\right)}_{inter-regional} + \underbrace{\boldsymbol{\mu}_i(t)}_{noise}$$

Noise followed an Ornstein-Uhlenbeck process. 5-minute-long runs (same duration as the empirical rs-fMRI acquisitions) were simulated for each subject using `neurolib` (C. Cakan, Jajcay, and Obermayer 2023). In order to compare model output and empirical rs-fMRI data, the firing rates of the excitatory populations are converted into estimated BOLD signals according to the Balloon-Windkessel model (Buxton et al. 2004).

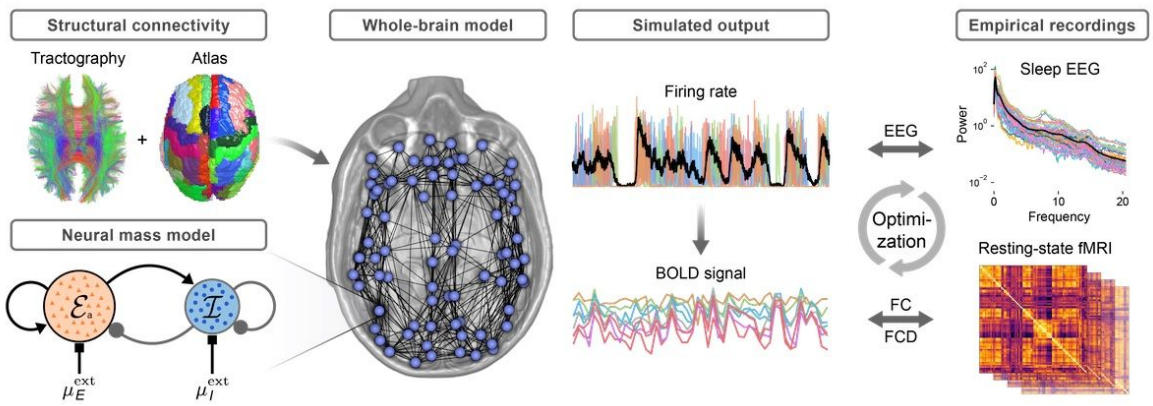

Figure 4: Overview of the construction of subject-level whole-brain models, and derivation of simulated BOLD signals. Adapted from C. Cakan, Jajcay, and Obermayer (2023)

3 Results

Whole-brain simulations of ROI-averaged resting state fMRI activity were successfully run for all 89 subjects using a an Intel Core i7-1280P laptop processor with 32 GiB of RAM. As reported by UBAS, the computation took about 52 minutes (using 16 threads in parallel).

```
>>> from neurolib.models.aln import ALNModel

>>> union = ubas.Cohort(set.union(patients, controls))

>>> union.compute(ALNModel, max_workers=16, store=True)
100%|██████████████████████████████| 89/89 [52:32<00:00, 37.55s/it]
```

Figure 5: Demonstration of the parallelized `compute()` interface, here employed to repeat the ALN simulation for all subjects using 16 simultaneous workers.

Visual inspection of the simulated functional connectivity (FC) for a particulary good patient is demonstrated in Figure 6. Both matrices are plotted together with the two structural connectivity matrices that generated the simulated dynamics (the length or "delay" matrix and the "fiber count" matrix).

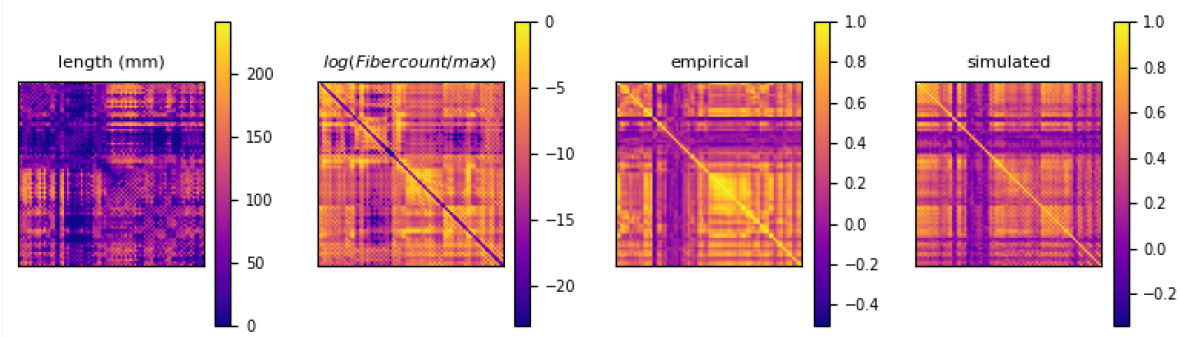

Figure 6: Structural and Functional connectivity matrices for one subject in the dataset. In the case of empirical and simulated functional connectivity, units are Pearson correlation values (dimensionless).

Between-matrix correlations for all FC pairs in the dataset produced an average correlation of only $r = 0.16$ between empirical and simulated data (Figure 7, second bar). This result was largely unchanged for a second simulation with individual structural connectivity replaced by the averages for each diagnostic group (Figure 7, third bar).

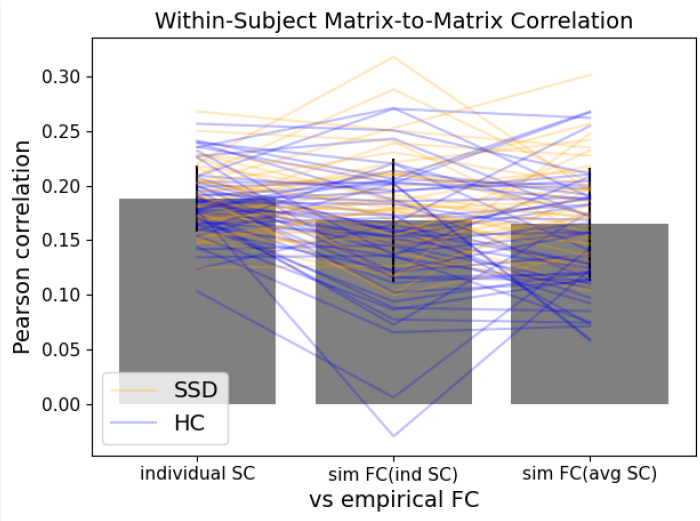

Figure 7: Between-matrix pearson correlations. First bar: empirical functional connectivity vs structural "fiber count" matrix. Second bar: empirical FC vs simulated FC (using individual SC). Third bar: empirical FC vs simulated FC (using group averages for SC).

The experiment was repeated for only five patients, but after optimizing the ALN coupling and noise parameter using a simulated annealing search and the between-matrix correlation as cost function. The new average correlation for these subset of subjects jumped to $r = 0.57$, and once again, using average structural connectivity did not statistically changed the results (Figure 8).

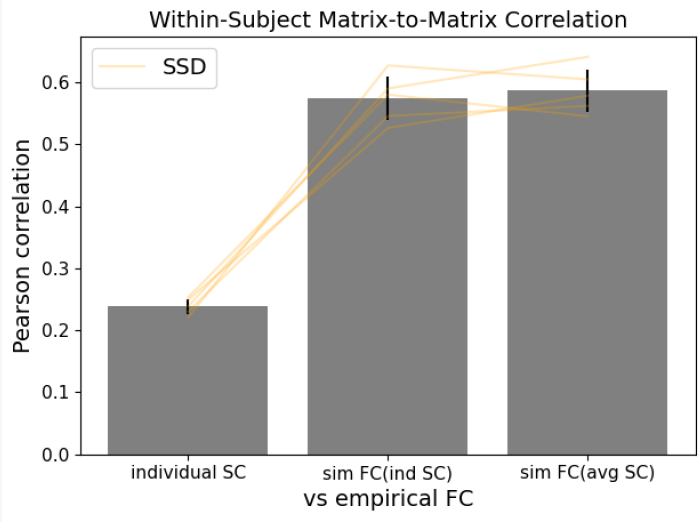

Figure 8: Between-matrix pearson correlations, after optimizing global coupling and noise parameters for a small subset of the dataset.

Finally, global brain connectivity (an average of mean FC correlations for all 100 cortical nodes) was computed for both groups using the empirical recordings. A previously-reported effect was observed (Metzner et al. 2024), whereby SSD patients exhibit lower global brain connectivity (Figure 9).

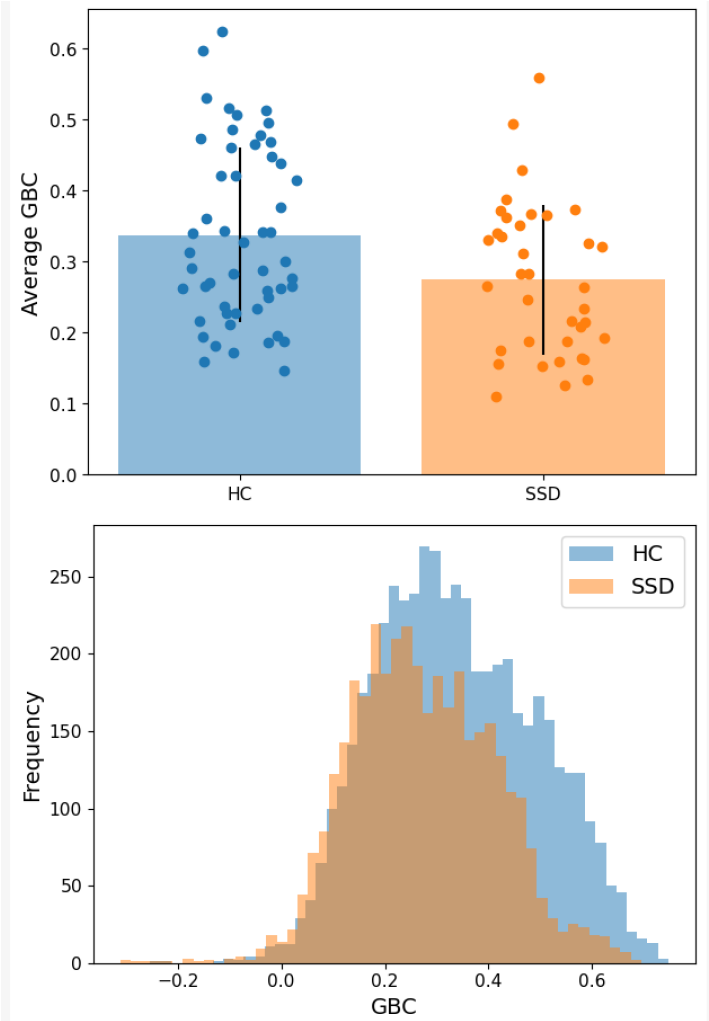

Figure 9: GBC for schizophrenia patients (SSD) vs controls (HC). Upper: subject average. Lower: GBC probability distributions from the GBC of every individual cortical ROI (all subjects at once).

```
>>> simulations = union.collect('quantities[ALNModel].simulated')
```

Figure 10: Demonstration of the `collect()` method to retrieve results of the ALN whole-brain model simulations for all subjects. The returned object is a lazy generator that can be iterated over when convenient.

4 Discussion

The present work introduced UBAS (Unified BIDS Automation at Scale), a Python library designed to bridge the gap between the organizational promise of the BIDS standard and the practical, often heterogeneous, needs of individual research analyses. By providing a scalable, declarative, and lazy-evaluated interface, UBAS allows researchers to manipulate complex neuroimaging datasets with minimal boilerplate code, while retaining full control over their analytical pipelines. The accompanying demonstration—fitting and simulating a whole-brain dynamical model for 89 subjects—serves as a concrete validation of UBAS's core capabilities: efficient data selection, parallelized computation, and seamless retrieval of results.

The primary contribution of UBAS is not a new analytical algorithm, but a meta-computational layer that standardizes and accelerates interaction with BIDS datasets. As neuroimaging pipelines grow in complexity and scale, the burden of data management increasingly distracts from scientific inquiry. UBAS addresses this by abstracting the filesystem hierarchy of BIDS into programmable Python objects (Cohorts, Subjects, Sessions, etc.) that behave like nested collections. Operations such as filtering participants, accessing specific modalities, or applying a function to every subject become concise, readable commands. This is exemplified in Figures 2, 5, and 10, where dataset subdivision, parallel simulation, and result collection are each achieved in one or two lines of code.

This approach complements existing ecosystem tools. Software like fMRIPrep, QSIPrep, XCP-D, and QSIrecon excel at standardized preprocessing, producing BIDS-derivatives that are themselves structured datasets. UBAS operates naturally on these outputs, enabling researchers to build custom post-processing, modeling, and analysis stages without reinventing data-loading logic. Future compatibility with foundational libraries like `nipype`, and `nilearn` is needed to ensure it can serve as the "glue" that integrates specialized tools into a coherent, reproducible

workflow. It must be noted that UBAS has some overlap with `nipype`, in the sense that both offer programmatic parallelization solutions for neuroimaging analysis. However, the two can also be used complementarily. `nipype` on one hand provides the critical connection to existing analysis software (e.g. AFNI, FSL, freesurfer, ANTS, etc.) that can be in turn exploited by UBAS. On the other, it also offers parallel processing capabilities in addition to a very powerful and general scheme to construct pipelines, where outputs from one external call are passed as inputs to the next one. UBAS is however oriented to data parallelism based on the BIDS hierarchy, and deep and complex `nipype` workflows should be easy to integrate with UBAS `.compute()` interface, with each BIDS member object performing a call to a deeper `nipype` workflow.

The whole-brain modeling demonstration served a dual purpose: it showcased UBAS's utility in a computationally intensive scenario and provided a scientific case study in dynamical system modeling of resting-state fMRI. The initial low correlation ($r = 0.16$) between empirical and simulated functional connectivity (FC) when using default ALN parameters is consistent with the literature, which often finds that simple anatomical coupling alone cannot fully explain empirical FC patterns. This underscores the complexity of the brain's dynamical regime and the role of factors not captured by the baseline model, such as regional heterogeneities in neural parameters, more nuanced coupling functions, or the influence of neuromodulators.

The significant improvement in correlation (to $r = 0.57$) after optimizing global coupling and noise parameters for a subset of subjects highlights the sensitivity of these models to parameter tuning. It suggests that a "one-size-fits-all" parameter set may be insufficient to capture individual variability in dynamics, which could relate to underlying neurobiological differences.

While UBAS presents a powerful paradigm, its current implementation has limitations and is still under heavy development before an official release. It is inherently tied to the BIDS specification; datasets requiring significant deviations may need adaptation. The lazy evalua-

tion and parallel processing are designed for single-machine, multi-core environments. Scaling to truly massive datasets (thousands of subjects) may require integration with cluster or cloud computing frameworks, which could be a future development path, perhaps leveraging Dask, Spark or Ray backends.

5 Conclusion

In summary, UBAS offers a practical solution to a growing problem in computational neuroscience: managing the complexity of large, standardized neuroimaging datasets without sacrificing analytical flexibility. By reducing the time and code required for data manipulation, it allows researchers to focus more intellectual effort on their core scientific questions—whether refining a complex model, as demonstrated here, or exploring entirely different analytical avenues.

The library's design philosophy—providing infrastructure rather than analysis—ensures its longevity and wide applicability. As the BIDS ecosystem continues to expand, tools like UBAS that promote interoperability, reproducibility, and scalable computation will become increasingly vital. The successful application to a whole-brain modeling pipeline underscores its readiness for use in demanding research contexts and invites the community to adopt, test, and extend its capabilities for their own diverse needs.

6 References

Aine, CJ, Henry Jeremy Bockholt, Juan R Bustillo, José M Cañive, Arvind Caprihan, Charles Gasparovic, Faith M Hanlon, et al. 2017. "Multimodal Neuroimaging in Schizophrenia: Description and Dissemination." *Neuroinformatics* 15 (4): 343–64.

Brett, Matthew, Christopher J. Markiewicz, Michael Hanke, Marc-Alexandre Côté, Ben Cipollini, Dimitri Papadopoulos Orfanos, Paul McCarthy, et al. 2024. "Nipy/Nibabel: 5.3.1." Zenodo.

Brette, Romain, and Wulfram Gerstner. 2005. "Adaptive Exponential Integrate-and-Fire Model as an Effective Description of Neuronal Activity." *Journal of Neurophysiology* 94 (5): 3637–42.

Buxton, Richard B, Kâmil Uludağ, David J Dubowitz, and Thomas T Liu. 2004. "Modeling the Hemodynamic Response to Brain Activation." *Neuroimage* 23: S220–33.

Cakan, Caglar, and Klaus Obermayer. 2020. "Biophysically Grounded Mean-Field Models of Neural Populations Under Electrical Stimulation." *PLoS Computational Biology* 16 (4): e1007822.

Cakan, C, N Jajcay, and K Obermayer. 2023. "Neurolib: A Simulation Framework for Whole-Brain Neural Mass Modeling. Cognitive Computation, 15 (4), 1132–1152."

Cieslak, Matthew, Philip A Cook, Xiaosong He, Fang-Cheng Yeh, Thijs Dhollander, Azeez Adebimpe, Geoffrey K Aguirre, et al. 2021. "QSIPrep: An Integrative Platform for Preprocessing and Reconstructing Diffusion MRI Data." *Nature Methods* 18 (7): 775–78.

Esteban, Oscar, Christopher J Markiewicz, Ross W Blair, Craig A Moodie, A Ilkay Isik, Asier Erramuzpe, James D Kent, et al. 2019. "fMRIPrep: A Robust Preprocessing Pipeline for Functional MRI." *Nature Methods* 16 (1): 111–16.

Gorgolewski, Krzysztof J, Tibor Auer, Vince D Calhoun, R Cameron Craddock, Samir Das, Eugene P Duff, Guillaume Flandin, et al. 2016. "The Brain Imaging Data Structure, a Format for Organizing and Describing Outputs of Neuroimaging Experiments." *Scientific Data* 3 (1): 1–9.

Gorgolewski, Krzysztof, Christopher D Burns, Cindee Madison, Dav Clark, Yaroslav O Halchenko, Michael L Waskom, and Satrajit S Ghosh. 2011. "Nipype: A Flexible, Lightweight and Extensible Neuroimaging Data Processing Framework in Python." *Frontiers in Neuroinformatics* 5: 13.

King, Maedbh, Carlos R. Hernandez-Castillo, Russell A. Poldrack, Richard B. Ivry, and Jörn Diedrichsen. 2019. "Functional Boundaries in the Human Cerebellum Revealed by a Multi-Domain Task Battery." *Nature Neuroscience* 22 (8): 1371–78.

Mehta, Kahini, Taylor Salo, Thomas J Madison, Azeez Adebimpe, Danielle S Bassett, Max Bertolero, Matthew Cieslak, et al. 2024. "XCP-d: A Robust Pipeline for the Post-Processing of fMRI Data." *Imaging Neuroscience* 2: 1–26.

Metzner, Christoph, Cristiana Dimulescu, Fabian Kamp, Sophie Fromm, Peter J Uhlhaas, and Klaus Obermayer. 2024. "Exploring Global and Local Processes Underlying Alterations in Resting-State Functional Connectivity and Dynamics in Schizophrenia." *Frontiers in Psychiatry* 15: 1352641.

Najdenovska, Elena, Yasser Alemán-Gómez, Giovanni Battistella, Maxime Descoteaux, Patric Hagmann, Se-

bastien Jacquemont, Philippe Maeder, Jean-Philippe Thiran, Eleonora Fornari, and Meritxell Bach Cuadra. 2018. "In-Vivo Probabilistic Atlas of Human Thalamic Nuclei Based on Diffusion- Weighted Magnetic Resonance Imaging." *Scientific Data* 5 (1): 180270.

Pauli, Wolfgang M., Amanda N. Nili, and J. Michael Tyszka. 2018. "A High-Resolution Probabilistic in Vivo Atlas of Human Subcortical Brain Nuclei." *Scientific Data* 5 (1): 180063.

Routier, Alexandre, Ninon Burgos, Mauricio Díaz, Michael Bacci, Simona Bottani, Omar El-Rifai, Sabrina Fontanella, et al. 2021. "Clinica: An Open-Source Software Platform for Reproducible Clinical Neuroscience Studies." *Frontiers in Neuroinformatics* 15: 689675.

Schaefer, Alexander, Ru Kong, Evan M Gordon, Timothy O Laumann, Xi-Nian Zuo, Avram J Holmes, Simon B Eickhoff, and B T Thomas Yeo. 2018. "Local-Global Parcellation of the Human Cerebral Cortex from Intrinsic Functional Connectivity MRI." *Cerebral Cortex* 28 (9): 3095–3114.

Yarkoni, Tal, Christopher J Markiewicz, Alejandro De La Vega, Krzysztof J Gorgolewski, Taylor Salo, Yaroslav O Halchenko, Quinten McNamara, et al. 2019. "PyBIDS: Python Tools for BIDS Datasets." *Journal of Open Source Software* 4 (40): 1294.